第5讲 · 软件系统分析与设计

架构设计

Software Architecture Design

从经典模式到智能体架构

架构模式 · 架构原型 · 部署策略 · 质量属性



谭火彬 潘海侠

北航软件学院 · 2026—2027学年 第1学期

00 本讲总览：一条从需求到结构的主线

设计基础
设计的目标、任务与方法
→
架构是什么
复杂性、定义、决策与视图
→
架构模式
七种典型模式与取舍
→
如何设计
ADD 过程、原型与选型
→
质量属性
性能、可靠、安全与权衡

前半程：想明白

为什么需要架构（从狗屋到摩天大楼）

软件设计与架构设计的概念

用 4+1 视图与 ADR 表达架构

后半程：做出来

架构模式选型与组合

属性驱动设计（ADD）与部署选型

质量属性场景评估与权衡

三个问题

1 架构为什么不可回避？

2 面对一类问题，该选哪种模式？

3 怎么证明架构满足质量要求？

能力落点 看得懂 · 选得对 · 画得出 · 评得准 · 写得清

成果对应 全部内容服务于作业 4：架构设计（15 分）

00 本章目标

01 讲清楚
能向他人解释软件设计与架构设计在开发过程中的位置、输入与输出

02 看得懂
能读懂分层、事件驱动、管道-过滤器、微内核、微服务、云与智能体架构图

03 选得对
能根据业务特征与约束，为系统选择合适的架构模式与部署策略

04 画得出
能用各种架构视图描述一个系统，产出逻辑视图与部署视图

05 评得准
能识别架构攸关需求，用质量属性场景评估方案的风险与权衡

06 写得清
能完成一份架构设计说明，含关键决策记录与选型理由

对应作业 4（15 分）：架构调研分析 5 分 + 系统架构设计 10 分

00 导入：从需求模型到架构蓝图

需求侧：我们知道了什么

用例模型
系统对外功能边界：检索成果、学者；认领成果

领域模型
学术成果、论文、专利、学者、领域

质量约束
1秒内返回搜索结果，学术成果数量不少于1000万

业务约束
数据质量如何保证、如何清洗?

两个团队、不同大模型技术、12月底上线

→

设计决策

分解 · 协作
技术 · 权衡

设计侧：我们要给出什么

架构视图
系统分成哪几块，块之间如何协作

接口契约
每块对外承诺什么能力、什么格式

部署视图
运行在哪些节点，如何伸缩与容灾

技术选型
框架、中间件、存储与云服务的选择

需求回答"做什么"，设计回答"分成几块、怎么协作、靠什么支撑"——中间隔着的正是架构决策

00 架构失败的代价：三类典型教训

扩展不上去

流量涨十倍就崩

所有功能打在一个进程里，只能整体复制

数据库单点、会话粘滞，横向扩展无效

架构根因：未做服务边界与无状态设计

替换不了

想换数据库要两年

业务规则写死在存储过程与 SQL 中，技术选型与业务逻辑深度耦合，无法局部替换

架构根因：缺少抽象层与依赖倒置

守不住

内部接口裸奔

服务之间默认互信，鉴权只在最外层做；一旦边界被绕过，数据直接暴露

架构根因：未设计信任边界与纵深防御

这三类问题都不是"代码写得不好"，而是"结构定错了"——改代码解决不了，只能改架构

01

理解软件设计

设计：把需求翻译成可实现的方案

设计的目标与定义

设计的四项内容

设计方法与设计原则

01 软件设计的目标：从"做什么"到"怎么做"

软件需求

解决"做什么"（What）

功能需求 · 非功能需求

设　计

映射（Mapping）

软件设计

解决"怎么做"（How）

架构 · 构件 · 接口 · 部署

设计是一种"映射"：把问题空间的需求，转换到解法空间的方案

软件设计的定义与任务

定义（IEEE 1990）

设计是"定义系统架构、构件、接口和其他系统特征的过程"，并得到"系统的执行结果"。

设计结果必须精确描述：

- 系统架构：系统如何分解为子系统与构件
- 构件：每个构件做什么、如何实现
- 接口：构件之间如何通信与协作
- 部署：构件如何映射到物理节点

软件设计的四项任务

架构设计
分解子系统、确定架构模式

构件（类）设计
设计类的属性、方法与关系

接口/用户界面设计
系统结构、界面布局和交互流程

数据设计
数据模型与持久化方案

本讲聚焦第一项：架构设计

01 软件设计的四项内容

① **架构设计**

系统分解为哪些构件、构件之间如何协作

产出：架构图、分层与模块划分

② **构件 / 模块设计**

每个构件内部的类、职责与协作关系

产出：类图、构件图、包图

③ **接口设计**

构件对外提供什么能力、遵循什么契约

产出：接口定义、API 文档、事件 Schema

④ **数据设计**

数据结构、数据库表、数据流与生命周期

产出：数据模型、表结构、ER 图

01 设计的输入与输出：设计不是凭空想象

输入

- 需求规格与用例模型
- 领域模型与业务规则
- 质量属性与约束（性能、安全、合规）
- 技术约束（既有系统、团队技能、预算）
- 组织与运维约束（部署环境、发布流程）

→ 设计活动

设计过程

分解 把系统切成可独立理解和实现的单元

抽象 忽略细节，定义稳定的对外契约

决策 在多个方案间权衡并记录理由

验证 用场景、原型或计算检验方案可行性

→ 设计产出

输出

- 架构视图（逻辑 / 开发 / 运行 / 部署）
- 接口契约与 API 文档
- 数据模型与存储方案
- 关键决策记录（ADR）
- 技术选型与部署方案

设计是迭代的：输出会反过来暴露需求的遗漏，形成"需求—设计—验证"的循环，而非一次性瀑布

01 软件设计方法：五个阶段，一条主线

结构化设计 1970s · 功能分解 › **面向对象设计** 1980s · 对象封装 › **基于构件设计** 1990s · 构件复用 › **面向服务 / 微服务** 2000s · 服务自治 › **智能体协同设计** 2020s+ · 自主协作

阶段	分解单位	核心抽象	解决什么问题	留下的新问题
结构化设计	函数 / 过程	模块、调用、数据流	把复杂功能自顶向下拆小，逻辑清晰可评审	以功能为中心，需求一变结构就崩
面向对象设计	对象 / 类	封装、继承、多态、接口	把"变化"封装进对象，贴合业务概念	抽象不当导致类爆炸，跨进程无效
基于构件设计	构件 / 包	端口、契约、二进制复用	把一组对象打成可独立复用的单元	构件的粒度与版本治理困难
面向服务/微服务	服务	自治、独立部署、契约优先	按业务能力拆分，独立演化与弹性伸缩	分布式一致性、治理与运维复杂度
智能体协同设计	智能体	目标、工具、记忆、协作协议	把"会推理、会调工具"的单元组织起来完成开放任务	结果不确定、成本与时延、可信与可控

不变的主线 不是推翻前者，而是同一思路的延伸：单元变大，契约不变

01 软件设计的作用：为什么"跳过设计"代价高昂

设计的三重作用

沟通的蓝图

让产品、开发、测试、运维对"系统长什么样"形成同一张图

决策的记录

把"为什么这样分、为什么选这个技术"固化下来，避免反复争论

变更的基线

后续需求变化时，能判断改动影响范围与代价

没有设计的四种症状

- ✕ 需求一变就"伤筋动骨"，改动像拆房子
- ✕ 关键知识只存在个别人脑子里，人走系统就瘫
- ✕ 无法估算工作量与风险，进度靠拍脑袋
- ✕ 质量属性失控：性能、安全、可用性只能事后打补丁

经验规律：缺陷发现得越晚，修复成本越高；架构级缺陷往往只能靠重写解决

设计不是"多画几张图"，而是把关键决策提前想清楚——设计的目标是"让后面的每一件事都更便宜"

01 架构的第一动作：把系统分解开

架构 = 分解：切成哪些单元 + 关系：单元之间如何协作　　不分解，就无法并行开发、无法度量、无法独立演化

沿四个维度决定"刀口往哪切"

按职责切
分层：接入、业务、数据各司其职

最常见，但层内仍可能是一团

例：Controller / Service / DAO

按变化切
限界上下文：变化一致的部分放一起

微服务拆分的首选依据

例：订单、库存、支付各自独立

按数据切
读写分离、按领域/租户分库

解决数据规模与一致性边界

例：订单库按用户 ID 分片

按团队切
康威定律：系统结构会贴合组织沟通结构

组织怎么分，架构往往就怎么长

例：两个团队 → 两个服务边界

切开就有代价，粒度是核心决策

分得细的收益
- 单元小、易理解，可独立开发与部署
- 故障隔离，单点问题不扩散
- 可按单元独立扩容与演化
- 团队可并行推进，交付节奏更快

分得细的代价
- 调用链变长：网络、重试、超时都要处理
- 数据一致性从"事务"变成"协调"
- 部署、监控、排障的单位成倍增加
- 团队间需额外沟通与契约维护
- 拆得太早，容易形成"分布式单体"

所以 架构设计的第一件事就是分解；而“分得好不好”不能凭感觉，有一定的设计原则

01 设计原则：判断"分得好不好"的六把尺子

高内聚

一个模块内部只做一件事，且做完整

架构层体现：按业务能力划分服务，而不是按技术层划分

低耦合

模块之间尽量少知道对方的细节

架构层体现：通过接口与事件通信，避免共享数据库表

关注点分离

不同性质的问题分开处理

架构层体现：业务逻辑不写在控制器或存储过程里

信息隐藏

变化的部分藏在稳定接口之后

架构层体现：封装第三方 SDK、数据库访问、模型调用

单一职责

一个单元只有一个被修改的理由

架构层体现：避免"上帝类"与万能服务

开闭原则

对扩展开放，对修改关闭

架构层体现：插件化、策略化，新增能力不改核心代码

这六条原则在后面的每一种架构模式里都会反复出现——它们才是"为什么这样分"的底层理由

02

理解架构设计

为什么需要架构：从狗屋到摩天大楼

建筑领域的启示

软件复杂性维度

软件架构的定义与重要性

从狗屋到摩天大楼（一）：搭建一间狗屋



一个人、一个周末就能建成的"建筑"

需要的技能与工具

构建者	一个人即可构建，自己动手、边建边改
建模	很少的建模——脑子里有想法就够了
过程	简单的过程：买材料、钉起来、完成
工具	简单的工具：锤子、锯子、钉子
失败代价	塌了就重搭，几乎没有不可逆的损失

需要的技能与工具

构建者	团队分工协作：设计、施工、监理各司其职
建模	设计（建模）先行：图纸必须先于施工
过程	一定的流程：基础 → 结构 → 装修，按工序推进
工具	强力的工具：起重机、标准构件、专业软件
失败代价	返工代价高昂，多数错误必须在图纸上消灭



规模一旦上台阶：先设计、后建造成为铁律

建一栋楼房：为什么团队必须"先画图"

盖楼房 vs 搭狗屋

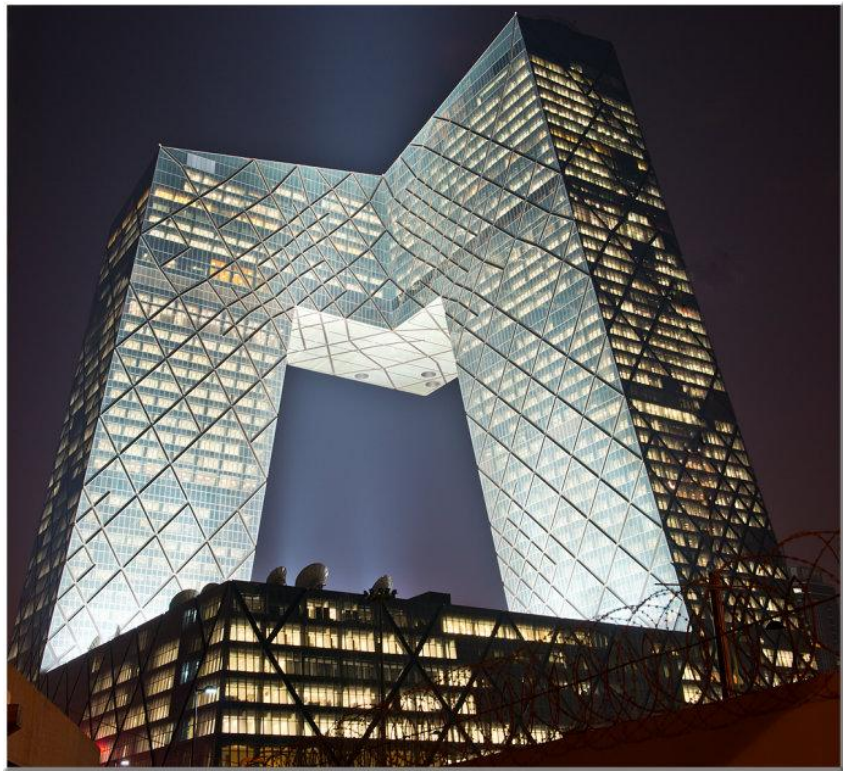
参与者	几十个工种、上百人协同
图纸	建筑、结构、水电、消防多套图纸
标准	材料规格、承重、抗震有强制规范
组织	按工序穿插施工，有总包与监理
验收	分部分项验收，问题不能留到交付

映射到软件：每一类"建造要素"都有对应物

建筑图纸	→	架构视图与设计模型
工种分工	→	模块划分与团队边界（康威定律）
材料与工艺标准	→	技术规范、编码规范、接口契约
施工组织与监理	→	CI/CD 流水线、代码评审、架构评审
分部分项验收	→	单元测试、集成测试、验收测试

楼房可以返工，但代价是工期与成本；
软件可以重构，但代价往往是用户流失与团队信心

从狗屋到摩天大楼（三）：搭建一座摩天大楼



高度与复杂度的极限挑战

系统级的设计问题

协同	多专业协同设计：结构、机电、幕墙、消防一体化
规范	严格的流程与规范：设计审查、施工监理、分级验收
系统设计	负载、抗震、消防、疏散——单靠经验无法覆盖
十倍法则	经验只能覆盖已知，大规模项目需要十倍以上的调研尝试
映射	软件同样如此：系统规模越大，架构越不可回避

02 建筑领域的启示：负载与失效

主要的负载

固定负载 建筑自重、结构重量

活动负载 人员、家具、设备

动态负载 风载、地震、车辆撞击

如何避免失效?

- 安全策略：设计冗余与安全系数
- 失效模式分析：哪里最可能先坏
- 规定结构负载极限并留有余量

映射到软件领域

固定负载 → 稳态业务流量

活动负载 → 日活用户、峰值并发

动态负载 → 秒杀、突发流量、热点事件

安全冗余 → 冗余部署、降级预案

架构 = 软件的"结构力学"：在负载与失效之间取得平衡

02 一套图纸不够：蓝图与架构图的共同点

建筑：为什么要多套图纸

- **平面图** 房间怎么排、动线怎么走
- **立面图** 外观长什么样
- **结构图** 梁柱承重怎么算
- **水电图** 管线怎么走、容量够不够
- **消防图** 疏散与报警如何设计

没有任何一张图能同时说清所有事——每种视图只回答一类问题

同理

软件：四类视图对应四类关切

- **逻辑视图** 系统分成哪些构件、职责是什么
- **开发视图** 代码怎么组织、模块怎么依赖
- **运行视图** 进程、线程、消息如何协作
- **部署视图** 跑在哪些节点、如何伸缩容灾
- **场景视图** 关键用例如何穿过这些结构

视图之间必须能相互印证，否则就是"图纸打架"

架构文档的价值不在于"图画得漂亮"，而在于让不同角色都能找到自己关心的那一面

02 软件复杂性的四个维度：架构要对付的到底是什么

复杂性来自哪里

技术复杂性 分布式、实时、高并发、异构集成

规模复杂性 百万行代码、数十个团队、跨地域协作

变更复杂性 需求、技术、组织与环境持续演化

约束复杂性 合规、安全、成本、时延互相牵制

大模型应用再叠一层：输出不确定、Prompt 成为资产、多模型与工具协作

复杂性带来的四个后果

看不懂
没人能完整说清系统如何工作，改动靠猜

改不动
一处修改牵连多处，回归测试成本剧增

测不全
组合爆炸，无法穷举场景，缺陷漏到线上

带不走
知识沉淀在人身上，团队流动即能力流失

架构的作用不是"消除复杂性"，而是把复杂性装进可控的边界

02 软件开发的六个驱动力

可扩展性 Scalability

用户与数据量增长时，系统能以合理成本扩容

当下：云与弹性伸缩让"扩容"从采购变成配置

可用性 Availability

关键业务要求 7×24 不间断，故障可秒级切换

当下：多活与容灾已成为互联网系统的默认要求

可恢复性 Resilience

故障发生时能降级、隔离、快速恢复

当下：分布式系统把"部分失败"当作常态来设计

可演化性 Evolvability

业务变化时能局部替换，而不必整体重写

当下：需求迭代周期从季度缩短到周，架构必须跟上

可观测性 Observability

能通过日志、指标、追踪定位问题

当下：微服务与智能体让调用链变长，黑盒不可接受

成本效率 Cost Efficiency

单位业务量的资源与人力成本可控

当下：Token 计费让"每次调用都有成本"成为设计约束

这些驱动力大多不是"功能"，而是"质量"——这正是架构要负责的部分

02 软件架构的定义：结构、关系与原则

三个经典表述

IEEE 1471
架构是系统在其环境中的基本组织结构，体现为构件、构件间关系，以及指导其设计与演化的原则。

Bass《架构实践》
架构是系统的结构，由软件元素、元素的外部可见属性以及它们之间的关系组成。

工程视角
架构是"一组做出后就很难再改的决策"，它把质量属性目标转化为结构约束。

四个常见误解

架构 ≠ 技术栈
选了 Spring Cloud 不等于有了架构，那只是材料

架构 ≠ 一张图
图是视图的投影，架构是背后的决策集合

架构 ≠ 全部设计
架构管构件与关系，类内部实现属于详细设计

架构 ≠ 一次成型
架构随认知与业务演进而迭代，但要控制变更成本

抓住三个关键词即可：构件（分几块）、关系（怎么连）、原则（凭什么这样分）

02 架构的重要性：五个"不可逆"的影响

01	**决定质量属性**	性能、可用性、安全性、可修改性主要由结构决定，代码优化只能在结构给定的上限内改善
02	**决定可演化性**	边界划对了，新需求是"加一块"；边界划错了，新需求是"拆一遍"
03	**塑造组织结构**	康威定律：系统结构会复制组织的沟通结构；架构反过来也约束团队如何划分
04	**决定复用与集成**	接口与边界清晰，才能被其他系统复用；否则只能复制粘贴
05	**支撑估算与风险管理**	有了构件划分与接口，才能拆分任务、估算工作量、识别技术风险

架构错了，代码再优雅也救不回来——架构设计是"一次决策、长期买单"的活动

02 架构是一组关键决策：用 ADR 把理由留下来

为什么引出 架构的本质是一组关键决策；而决策最容易丢失的不是结论，是当时的约束、备选方案与代价——半年后只剩结果，没人说得清为什么

ADR 是什么

Architecture Decision Record（架构决策记录）：用一页纸记录一个架构决策——当时的问题与约束、考虑过的选项、最终决定、以及要承受的后果。

一份 ADR 的五个部分

- **标题** 一句话说明决策内容，编号可检索
- **背景** 面临什么问题、有哪些约束与力量
- **决策** 我们决定做什么（明确、可执行）
- **状态** 提议 / 已接受 / 已废弃 / 被取代
- **后果** 好处、代价与对后续决策的约束

一个决策一个文件，放进代码仓库，随代码一起评审、追溯与演进

示例 ADR-007 · 已接受

订单与库存采用事件驱动解耦

背景：大促下单峰值是日常 30 倍，同步扣减库存导致订单服务被拖垮。

备选：① 同步调用 + 线程池扩容；② 下单落库后异步扣减（选中）；③ 预扣库存 + 定时对账。

决策：下单只写订单库并发布事件；库存服务异步消费并预占，失败则补偿取消。

后果：下单时延降到 80ms；但需接受最终一致，并增加消息中间件、幂等与对账成本。

半年后新人问"为什么不用同步"，答案就在这个文件里；若换成同步，第 3 条备选会被重新翻出来评估

哪些决策必须写

- 影响结构且难以回退
- 跨团队/跨系统
- 存在明显分歧
- 引入新技术或外部依赖
- 违反常规、需要解释

02 架构视图（一）：一个系统，多种看法

为什么必须分视图

- 一个系统太大，任何单张图都会"信息过载"
- 不同干系人关心的侧面完全不同
- 不同关注点混在一张图里，会互相干扰、无法验证
- 视图是"投影"：每个视图只暴露与某类决策相关的元素

视图不是越多越好：能回答关键问题、彼此自洽即可

四类视图与它们回答的问题

逻辑视图 系统提供哪些能力、由哪些构件实现
主要读者：产品、业务、架构师

开发视图 代码如何组织、模块如何依赖、如何构建
主要读者：开发、构建工程师

运行视图 运行时有哪些进程与线程、如何通信与并发
主要读者：开发、性能工程师

部署视图 软件映射到哪些物理/云节点、如何伸缩容灾
主要读者：运维、SRE、安全

再加"场景视图"把四者串起来：用关键用例验证结构是否真的能跑通

02 架构视图（二）：4+1 视图与 UML 的对应



四个视图 + 一个场景，场景负责验证与串联，故称 4+1

4+1 不是唯一体系：C4 模型、SEI 三视图与现代补充视图

见下一页

每个视图落到哪种 UML 图

逻辑视图	类图、对象图、包图	构件、职责与静态关系
开发视图	构件图、包图	代码组织与依赖
运行视图	顺序图、活动图、状态图	交互、并发与状态
部署视图	部署图	节点、制品与网络
场景视图	用例图、顺序图	关键场景的端到端走查

02 架构视图（三）：4+1 之外，还有哪些视图体系

C4 模型：按"放大"组织

Simon Brown 提出，适合敏捷团队的轻量体系

C1 上下文	系统与用户、外部系统的关系
C2 容器	应用、服务、数据库、消息队列
C3 组件	容器内部的分组与职责
C4 代码	类级细节，通常由工具生成

优点：一套图能讲清"系统在哪、由什么组成、里面怎么分"，非技术读者也看得懂

SEI 视图：按"结构类型"分

卡内基梅隆 SEI 的视图分类，比 4+1 更细

模块视图	分解、使用、分层、泛化：代码静态结构
构件-连接件视图	进程、线程、C/S、发布-订阅：运行时结构
分配视图	部署、实施、团队分工：软件到哪里去

与 4+1 的对应：逻辑/开发≈模块视图，运行≈构件-连接件视图，部署≈分配视图

价值：把"分配"单独拎出来，提醒架构必须落到人、机器与网络

现代系统常补的视图

数据视图	领域模型、数据分布、主数据与血缘
安全视图	信任边界、认证授权、密钥与审计
可观测视图	日志/指标/追踪埋点与告警链路
数据流/集成视图	系统间接口、事件主题与契约
成本视图	资源配额、扩容单价、Token 成本
智能体视图	上下文组装、工具调用链、记忆与评测

这些视图不替代 4+1，而是在其上做"专题切片"

怎么选视图

三步走

① **先定读者**
给业务、开发还是运维看

② **再定问题**
这张图要支撑哪个决策

③ **保持自洽**
同一元素在各视图中名字与边界一致

02 架构与需求：质量属性才是架构的真正输入

功能性需求

决定"系统做什么"，影响构件的划分依据——按业务能力分，还是按处理阶段分。

质量属性（ASR）

决定"做到什么程度"，影响构件的组织方式与技术选择——这才是架构师最该盯住的输入。

同一个功能，不同的质量要求会导出完全不同的架构

需求表述	架构含义	典型战术	可能选用的模式
支持 5000 人同时在线查询	高并发、可水平扩展	负载均衡、缓存、读写分离	分层 + 云弹性
大促期间不能宕机	高可用、可降级	冗余、熔断、限流、多可用区	微服务 + 多活部署
三个月内上线，之后快速试错	低初期成本、易改	模块化单体、特性开关	分层（先不拆服务）
要接入第三方模型与工具	外部依赖多、结果不确定	适配层、超时重试、人工兜底	智能体架构 + 网关

架构师的核心能力：把模糊的质量期望，翻译成可检验的场景，再翻译成结构决策

03

架构设计模式

从成熟模式出发，而不是从零开始

分层 | 事件驱动 | 管道-过滤器 | 微内核 | 微服务 | 虚拟化 / 云 | 智能体架构（新）

03 架构模式：站在巨人的肩膀上

- 架构不是从零开始设计的：同类系统的参考架构有很大的借鉴意义
- 架构模式：由软件架构师在长期实践中沉淀、被反复验证的通用解决方案
  - 模式回答三个问题：构件如何划分、如何交互、职责如何分配
  - 选型要点：没有最好的模式，只有最匹配问题与约束的模式

典型架构模式一览

分层架构	企业级应用的标准模式，职责分层
事件驱动	异步分布式，高可扩展性
管道-过滤器	数据依次流过处理步骤，如编译器
微内核	核心 + 插件，面向产品化应用
微服务	服务独立部署、独立演化
虚拟化 / 云	弹性伸缩，应对高并发
智能体架构（新）	大模型驱动的自主协作系统

03 典型架构模式全景：本讲重点讲的七种

01 分层架构

按职责纵向切分，上层依赖下层

典型：企业业务系统、信息系统

02 事件驱动

构件通过事件异步通信，解耦生产与消费

典型：订单履约、风控、IoT

03 管道-过滤器

数据依次流过处理步骤，每步可独立替换

典型：编译器、日志/ETL、推理流水线

04 微内核

最小核心 + 插件扩展

典型：IDE、浏览器、规则引擎

05 微服务

按业务能力拆分为自治服务，独立部署与演化

典型：大型互联网平台、SaaS

06 虚拟化 / 云

应用与资源解耦，按需弹性伸缩

典型：流量波动大、快速上线

07 智能体架构

大模型驱动，规划 + 工具调用 + 记忆 + 协作

典型：智能助手、编码智能体、自动化流程

每个模式三问

① 怎么分？——构件与职责

② 解决什么？——适用问题

③ 有什么代价？——约束与风险

模式一：分层架构（即 N 层架构）

概念

- 最常用的架构模式，大部分企业级应用的标准模式
- 符合传统开发习惯与组织结构，各层各司其职
- 上层只能调用相邻下层，单向依赖



03 分层架构实例：一个电商系统怎么分层



▲ 上层可调用下层

✕ 禁止跨层与反向依赖

三条落地规则

单向依赖

只允许上层依赖下层；下层通过接口回调上层（依赖倒置）

层内高内聚

同一层的构件职责同质，避免"什么都往里放"

跨层要显式

确实需要跨层时，用门面或防腐层包装，别直接穿透

反模式：分层过多（7 层以上）导致每加一个功能要改 7 个文件——分层是为了隔离变化，不是为了好看

03 分层架构：优势、适用性与选择

优势

- 组件各司其职、相互分离，只属于某个特定层面
- 层内替换不影响其他层（换数据库、换界面框架）
- 符合团队分工：前端、后端、数据各配专人
- 技术成熟、生态完善，招聘与维护成本低

注意

- 层层传递导致性能损耗，修改一个功能要贯穿多层
- 容易演化成"贫血"的增删改查应用，业务逻辑散落各层
- 部署是整体的，无法按层独立伸缩

适用性

- 标准化业务线应用程序，功能上不限于增删改查
- 需要快速构建、团队规模有限的项目
- 业务规则清晰、变更频率中等的系统
- 典型例子：教务系统、订单系统、内部管理平台

模式二：事件驱动架构（一）概念与拓扑

概念

- 流行的异步分布式模式，可生成高可扩展性应用
- 适应能力强，可用于小程序，也可用于大型复杂系统
- 构件之间不直接调用，通过"事件"进行通信
- 两种主要拓扑结构：中介人拓扑 与 代理拓扑

现代落地形态：消息队列（如 Kafka）+ 发布/订阅，是微服务协作的主流方式

中介人拓扑（Mediator）

事件生产者 ▶ 事件中介人 ▶ 事件消费者

中介人统一编排事件处理顺序

适合流程可控、需要集中管理的场景

代理拓扑（Broker）

事件生产者 ▶ 事件代理链 ▶ 级联处理

无中心编排，代理链级联处理事件

适合高吞吐、松耦合的场景

事件驱动架构（二）：中介人拓扑详图



特点：处理顺序由中介人集中控制，便于监控与治理；代价是中介人可能成为单点与瓶颈

事件驱动架构（三）：代理拓扑与适用性

代理拓扑：链式级联处理



- 每个消费者处理完发布新事件，形成处理链
- 无中心编排，吞吐量高、耦合最低
- 代价：全链路监控与事务一致性更难

优势与适用性

- 容易适应复杂且混乱的环境，天然可扩展
- 适合异步、高并发、组件需要独立演化的系统
- 典型例子：订单履约、风控流水线、物联网数据流
- 挑战：调试困难（事件链路长）、最终一致性、重复消费需幂等

03 事件驱动实例：一次下单，六次事件

事件通道（消息中间件）：订单已创建 · 库存已预占 · 支付已成功 · 已发货

1. **下单**
   订单服务写入订单，发布「订单已创建」
2. **库存预占**
   库存服务消费事件，预占成功后发布「库存已预占」
3. **发起支付**
   支付服务消费事件，调用支付网关
4. **支付成功**
   发布「支付已成功」，订单状态推进
5. **履约发货**
   仓储服务消费事件，生成拣货任务
6. **通知用户**
   通知服务消费事件，发短信/推送

这样设计换来了什么

下单只需 1 次数据库写入；新增"积分服务""风控服务"只需订阅事件，不改订单代码

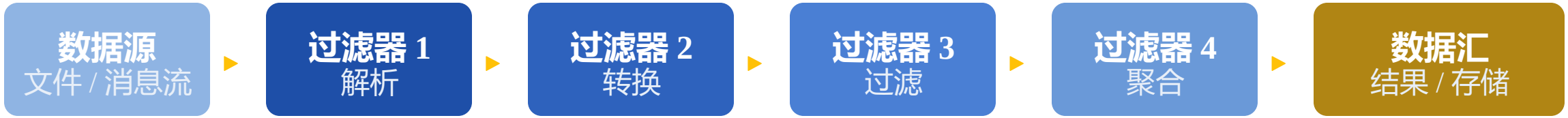
必须补上的三件事

消费幂等（去重表）· 失败补偿（重试 + 死信队列）· 全链路追踪（事件 ID 贯穿）

事件驱动的本质：把"调用"换成"通知"，把"同步等待"换成"最终一致"

03 模式三：管道-过滤器——让数据流过一条加工线

结构：数据在独立的过滤器之间经管道单向流动



要点：过滤器相互独立、无需知道上下游是谁；增量消费输入、增量产出输出，因此天然可并行

典型实例

编译器	词法分析 → 语法分析 → 语义分析 → 代码生成
Unix 管道	cat log | grep ERROR | sort | uniq -c
数据流水线	采集 → 清洗 → 转换 → 入仓（ETL）
流处理	Kafka / Flink：事件流经多级算子实时加工

设计要点：为每个过滤器定义清晰的数据契约；管道上设置缓冲与背压，避免慢消费者拖垮整条流水线

优势

过滤器可复用可替换、易组合；每个过滤器可独立测试；吞吐量大，天然支持并行与批处理

代价与适用

不适合交互式应用；步骤间传数据有开销，端到端时延由最慢一级决定；适合"数据流加工"型系统

03 模式四：微内核架构（插件架构）

概念

- 基于产品打包交付的应用系统的自然选择
- 把最小功能核心与扩展功能分离，通过插件扩展能力
- 核心系统稳定，插件按需装卸、独立演化

优势

- 核心小而稳定，故障面小
- 扩展能力强，第三方可开发插件
- 适合功能随客户而变的场景



例：Eclipse 插件体系、VS Code、浏览器扩展、IDEA 插件、低代码平台连接器

03 微内核实例与权衡：核心要多小才算小

四个真实例子

IDE	核心负责编辑、构建、调试；语言支持、版本控制、主题皆为插件
浏览器	核心是渲染引擎与安全沙箱；扩展通过标准 API 接入
规则引擎	核心是规则执行器；业务规则以配置或脚本形式热插拔
SaaS 产品	核心是通用能力；客户定制以插件/扩展包交付，不 fork 主干

判断标准：把"会随客户/场景变化"的部分全部赶出核心

代价与约束

! **核心 API 必须稳定**
插件依赖核心接口，接口一改全部插件失效，需要版本化与兼容策略

! **插件治理复杂**
加载顺序、依赖冲突、权限边界、生命周期管理都要设计

! **隔离性有限**
同进程插件互相影响，崩溃会拖垮核心；需考虑沙箱或独立进程

! **调试链路变长**
问题可能出在任意插件，需要插件级日志与追踪

微内核的适用前提：产品要"一套核心、多客户定制"，且变化集中在功能扩展而非核心流程

03 模式五：微服务（一）从单体到服务化的演进

2000s 前
单体应用
一个进程、一个数据库、一次部署
简单，但耦合

›

2000s
面向服务 SOA
以 ESB 集成企业内异构系统
复用，但总线沉重

›

2014 后
微服务
按业务拆分、独立部署、去中心化治理
自治，但分布式复杂

›

现在
服务网格 / Serverless
服务治理下沉到基础设施，按调用付费
轻量，但可观测要求高

微服务的四个本质特征

- 按业务能力拆分，而非按技术层拆分
- 每个服务独立部署、独立伸缩、独立演进
- 服务间通过轻量协议通信（REST / gRPC / 事件）
- 去中心化治理：每个团队对自己服务全生命周期负责

什么时候不该用微服务

团队规模小、领域边界不清、运维能力不足、对一致性要求极高且无法接受最终一致时，模块化单体往往是更理性的选择。

架构的第一原则是匹配组织与阶段，而不是追逐技术潮流

03 模式五：微服务（二）按什么拆——限界上下文

电商系统的限界上下文

订单上下文
订单、订单项、状态机

库存上下文
库存、预占、盘点

支付上下文
支付单、退款、对账

商品上下文
SPU、SKU、类目

会员上下文
账户、等级、积分

履约上下文
拣货、发货、签收

限界上下文 = 一个词在这里只有一个含义。"商品"在商品上下文是SPU/SKU，在订单上下文是"下单时的快照"

拆分的四条准则

业务能力内聚
一个服务内改动原因单一，能独立交付一个业务价值

数据自治
每服务独享数据库，禁止跨库联表查询

契约稳定
对外接口按版本演进，内部实现可自由替换

团队可承载
一个服务由一个 5~9 人团队负责（两个披萨原则）

常见错误：按"controller / service / dao"拆服务，或按"每个表一个服务"拆——这叫分布式单体

03 模式五：微服务（三）治理体系与四个硬骨头

配套治理能力：拆得开，还要管得住

注册与发现
服务实例自动注册，调用方按名寻址

API 网关
统一入口，做认证、路由、限流

配置中心
配置集中管理，支持热更新与灰度

熔断限流
故障隔离，防止雪崩

链路追踪
一次请求跨服务全链路可视

可观测
指标、日志、追踪三位一体

分布式事务

跨服务写入无法用一个本地事务保证

应对：Saga 补偿、TCC、本地消息表、最终一致

数据一致性

跨库查询与统计变难，读模型滞后

应对：事件驱动同步、CQRS 读写分离、对账兜底

服务依赖失控

调用链路过长，一个慢服务拖垮全局

应对：依赖分级、超时预算、舱壁隔离

测试与发布

本地跑不起来，联调成本高

应对：契约测试、服务虚拟化、灰度与金丝雀

微服务不是"免费的可扩展性"：它把复杂度从代码里搬到了基础设施与组织协作上

模式六：虚拟化 / 云架构

概念

- 解决高可扩展性与高并发问题
- 适用于用户规模变化大、不可预见的系统
- 应用与基础资源解耦：按需申请、按量计费、弹性伸缩
- 典型形态：容器化部署 + 弹性伸缩组 + 负载均衡

本课程实践即部署在华为云：云上容器与中间件直接支持该模式的落地



03 云原生：容器、编排与无服务器

三层服务模型

IaaS 虚拟机、网络、存储，自己装系统与中间件

PaaS 托管数据库、消息队列、容器平台，只管应用

SaaS 直接使用软件服务，按订阅付费

容器 vs 虚拟机：容器共享内核，启动秒级、镜像小；虚拟机隔离更彻底，适合强合规场景

云原生的四个支柱

容器化 应用与依赖打包成不可变镜像，环境一致

编排调度 Kubernetes 负责部署、扩缩容、自愈、滚动升级

微服务化 应用拆分为可独立部署的服务单元

持续交付 流水线自动化构建、测试、发布与回滚

Serverless：把"服务器"也交给平台

按事件触发、按调用计费、自动伸缩到零；适合突发流量与轻量任务，代价是冷启动与厂商绑定

对应本课程实践：华为云 + 云码 CodeArts + AI 编程工具，正是"云原生 + 智能辅助"的完整链路

03 模式七：智能体架构总览——一个智能体由哪些部件构成

1	**交互入口**	对话 / 表单 / 语音 / API；输入归一化与结果呈现
2	**编排控制（Agent Loop）**	任务分解、下一步决策、状态推进、终止判断
3	**模型推理（LLM 网关）**	模型路由、推理调用、结构化输出、缓存与降级
4	**能力供给**	工具调用（MCP / 函数）、检索（RAG）、业务系统 API
5	**记忆与状态**	工作记忆、会话状态、长期记忆、向量与 KV 存储

横切·治理 权限·护栏·审计·评测·成本

横切·可观测 Trace·指标·成本·回放

五层各自的职责边界

交互入口	只做协议与体验，不写业务判断
编排控制	决定"下一步做什么"，是智能体的大脑皮层
模型推理	提供语言理解与生成，不做流程控制
能力供给	把外部世界封装成可调用、可审计的契约
记忆状态	让智能体"记得住、接得上"，且可被清除

智能体 ≠ 一个大模型

它是"模型 + 上下文 + 工具 + 控制循环 + 治理"的组合；单智能体与多智能体共用同一套部件，差别只在"有几个决策中心、谁来协调"

判断标准：能用确定性规则稳定解决的，不要用智能体；智能体适合"规则写不全、需要理解与判断"的任务

03 单智能体架构：一次任务是怎么跑完的

运行时结构：装配 → 决策 → 执行 → 观察 → 收敛

① **上下文装配** 系统提示 + 任务 + 工具清单 + 检索片段 + 历史摘要 + 记忆

② **推理与决策** LLM 判断：直接回答，还是调用某个工具？产出结构化动作

③ **工具执行** 在受控环境调用 API / 检索 / 代码，校验参数与权限

④ **观察回填** 结果裁剪、脱敏后写回上下文；失败则带错误信息重试

⑤ **收敛与写回** 满足终止条件则输出；把要点写入记忆，供下次复用

②→③→④ 构成一个循环（ReAct）：每一步都是一次"想—做—看"，直到给出最终答案或用完预算

各组件的设计要点

上下文装配器	按预算分配 token：工具定义 > 任务 > 检索 > 历史；宁可摘要，不要硬塞
决策器	输出必须是结构化动作（工具名 + 参数），而不是自由文本
执行器	每个工具独立超时、重试、幂等键；失败要回传可读错误
状态机	明确"进行中/待确认/已失败/已完成"，支持暂停与恢复
终止策略	最大步数 + 时间预算 + 成本上限 + 无进展检测，四者缺一不可

实现策略（选一种起步）

ReAct：边想边做，适合探索型任务，步数多、成本高

Plan-and-Execute：先出计划再逐条执行，适合步骤清晰的流程

固定工作流：把确定性步骤写成代码，只在关键节点调模型（最省最稳）

03 多智能体架构：多个决策中心如何组织

编排 / 协调者 任务分解、角色分派、进度跟踪、结果汇总与冲突裁决

规划智能体	检索智能体	执行智能体	评审智能体
独立上下文	独立上下文	独立上下文	独立上下文

共享状态 / 黑板
任务板、中间产物、决策记录

通信通道
消息队列、A2A 协议、MCP 工具共享

共享执行底座 工具池与注册中心、沙箱环境、企业系统 API、模型网关（各智能体共用，避免重复建设）

治理与可观测 全局预算、权限边界、工具审计、跨智能体 Trace（一次任务的完整调用树）

关键判断：多智能体不是"能力更强"，而是"用并行与分工换上下文与专注度"，代价是协调成本与不确定性

三种拓扑：谁来决定下一步

中心编排
一个 Supervisor 分解任务、分派、汇总；成员之间不直接对话

优势：流程可控、易审计、易追责
代价：编排者成瓶颈与单点，能力上限受限于它

层级编排
Supervisor 下再设子编排者，形成"总包—分包"两级结构
优势：适合大型任务、可按域拆分
代价：层级深、通信开销与延迟上升

去中心协作
智能体直接对话/辩论，或通过黑板协商，无固定指挥
优势：灵活、抗单点、易产生涌现
代价：难收敛、成本高、结果难复现

先问：真的需要多智能体吗
单体够用就不拆；只有"上下文装不下、角色需隔离、任务可并行"三者之一成立时，拆分才划算

03 组件详解①：模型接入层与 LLM 网关

它负责什么

统一接入	所有模型调用走同一入口，业务不直连 SDK
模型路由	按任务难度、时延、成本选模型；失败可升级
配额限流	按租户/用户/应用分配 QPS 与 token 预算
缓存	相同请求命中结果缓存或语义缓存
降级容灾	主备模型切换、超时熔断、返回兜底答案
计量审计	记录 token、费用、调用方，支持对账

设计要点

分层路由	简单分类用小模型，复杂推理才升级到大模型
结构化输出	用 JSON Schema 约束格式，解析失败自动重试一次
超时与重试	按任务设超时；重试要带幂等键，避免重复副作用
流式优先	首 token 时延决定体验，长输出必须流式返回
上下文长度	超长输入先摘要/分段，不要依赖"塞得下"
避免锁定	统一抽象层，模型可替换；私有化与公有云同协议

可用技术

接入协议	OpenAI 兼容 API（事实标准）、gRPC；SSE 流式
推理引擎	vLLM、SGLang、TensorRT-LLM（私有化高吞吐）
统一网关	LiteLLM、One-API、Envoy AI Gateway、Higress
语义缓存	GPTCache、Redis Vector、云厂商 Prompt Cache
结构化	Function Calling、JSON Schema、约束解码
可观测	OpenTelemetry GenAI 语义约定、Langfuse

一条硬规则：业务代码里不允许出现模型 SDK——所有调用必须经过网关，才能可替换、可计量、可治理

03 组件详解②：上下文装配与记忆

上下文装配：先算预算，再拼内容

系统提示	角色、边界、输出规范（10%）
任务与约束	当前目标、验收标准、禁忌（10%）
工具定义	名称、用途、参数、示例（15%）
检索片段	知识库/文档，带来源与时间（30%）
对话历史	滚动摘要 + 最近若干轮原文（25%）
预留空间	给模型思考与输出，别占满（10%）

压缩顺序：先摘要、再按轮淘汰、最后才截断；关键事实用结构化槽位保留

记忆分层：写什么、何时写、怎么取

工作记忆	当前任务的计划、中间产物、待办；任务结束即清
情景记忆	过去发生过什么：对话、操作、结果与反馈
语义记忆	沉淀下来的知识与用户偏好，跨会话复用
程序记忆	可复用的技能与提示模板，如"周报生成套路"

写入策略

任务结论必写、用户偏好即时写、失败教训写；写前查重，写后打置信度与时间戳

读取：语义相似度 + 时间衰减 + 重要性加权，避免"旧偏好压过新需求"

检索增强：五步流水线

1. 切分：按标题/语义分块，10–15% 重叠
2. 向量化：Embedding 模型统一版本
3. 混合检索：BM25 + 向量，各取所长
4. 重排：Rerank 取前 5–8 条
5. 注入：带来源、页码与时间，便于引用

常见技术

pgvector / Milvus / Qdrant / Elasticsearch

bge-m3、bge-reranker、Cohere Rerank

Mem0 / Letta / Zep（记忆层）、Redis（会话）

检索质量看三项

命中率（召回是否够）、引用准确率（是否答对）、上下文占比（是否浪费）

三个坑　塞满：关键信息被埋在中间，模型反而看不见　污染：错误结论被写入记忆，之后反复强化　越权：检索跨租户/跨用户召回，造成数据泄露

03 组件详解③：工具与执行

工具抽象：给模型看的 API 文档

名称	动词 + 对象，如 create_order、search_docs
描述	一句话说清用途、何时用、何时不要用
参数	JSON Schema：类型、必填、枚举、示例值
返回	结构化结果 + 错误码；长结果分页或摘要
边界	权限范围、数据范围、调用频率上限
版本	参数变更要向后兼容，废弃需公告

描述写不好，模型就不会用：工具选择错误的根因，多半在描述而不是模型能力

调用协议与工具网关

Function Calling	模型输出结构化调用意图，运行时执行并回填结果
MCP	标准化工具/资源接入：本地 stdio、远程 HTTP + SSE
OpenAPI 注册	把现有 REST 接口自动注册为工具，复用既有治理
工具网关	注册、鉴权、限流、参数校验、审计的统一入口
A2A	智能体之间的协作协议，与 MCP（智能体→工具）互补

分层建议：模型只看见"工具目录"，网关负责路由到具体实现——换实现不改提示词

执行安全与可靠性

最小权限	以用户身份透传，禁止用超级账号代跑所有请求
沙箱隔离	代码执行用 E2B / Firecracker / gVisor；网络白名单
幂等补偿	写操作带幂等键；失败可重试、可回滚、可对账
超时熔断	单工具独立超时；连续失败熔断并降级
审计留痕	记录调用者、上下文、参数、结果与耗时
人工审批	高风险动作（转账、删除、外发）必须人工确认

工具设计四原则　少而清晰：一个工具做一件事　可验证：有明确成功/失败判据　可回滚：写操作能撤销　最小权限：只给必要的范围

03 组件详解④：编排控制、治理与可观测

编排控制：把"循环"画出来

固定工作流	步骤确定：代码编排为主，只在关键节点调模型（最稳最省）
ReAct 循环	想—做—看，边探索边推进；需严格步数与预算
Plan-Execute	先出完整计划，再逐步执行；计划可人工审阅
图 / 状态机	节点是步骤，边是条件；分支、并行、回退都能表达
检查点	每步持久化状态，支持暂停、恢复、重放与人工接管

终止条件四件套：最大步数 · 时间预算 · 成本上限 · 无进展检测（连续两步结果相同即停）

治理与护栏：让行为可约束

输入护栏	提示注入检测、越权意图识别、敏感信息脱敏
输出护栏	格式校验、事实性核对、敏感内容过滤、引用可溯源
工具权限	按角色授权；高风险动作强制人工审批（人在回路）
预算控制	按任务/用户设 token 与费用上限，超限熔断
数据合规	最小化留存、可删除、可导出；满足审计与备案要求

可观测与评测：让效果可证明

Trace	一次任务一棵调用树：模型、工具、耗时、token、错误
指标	成功率、平均步数、P95 时延、单任务成本、人工介入率
离线评测	固定测试集 + 回归对比；规则断言 + LLM 评分 + 人工抽检
线上验证	采样回放、灰度 A/B、badcase 归因到具体节点
版本管理	提示词、工具、模型、索引都要有版本号，可回滚

常见技术

LangGraph、Temporal、Dify 工作流、OpenTelemetry GenAI

Langfuse / LangSmith、Guardrails / NeMo Guardrails、OPA

上线门槛：没有 Trace、没有评测集、没有人工兜底的智能体，不要进生产环境

03 选择与组合：先单体跑通，再按瓶颈拆

形态	适用场景	优势	关键约束
单体智能体	单一领域、流程清晰的助手与编码助手	简单可控、成本低、易调试	上下文有限，复杂任务易失控
固定工作流 + 节点智能体	步骤确定、局部需要判断的业务流程	最稳最省，可审计	灵活性有限，流程变更要改编排
多智能体·中心编排	多角色流水线（调研→写作→审核）	分工清晰、过程可控	编排者复杂，单点风险
多智能体·去中心	开放探索、辩论收敛类任务	灵活、能力易涌现	难收敛，Token 成本高
RAG 增强	企业知识问答、文档与客服场景	可控可溯源、知识易更新	依赖知识库与检索质量

三个问题定形态

① 任务能拆成可并行的子任务吗？→ 不能：单体；能：考虑多智能体

② 角色需要隔离上下文与权限吗？→ 需要：多智能体分工

③ 单个上下文装得下全部信息吗？→ 装不下：拆分 + 共享黑板

三问都答"否"，就别拆——多数场景单体 + 好工具已经够用

组合原则：寄生而非重造

微服务负责承载与治理，事件驱动负责解耦协作，云负责弹性伸缩，RAG 与工具层负责能力供给；纯智能体系统反而少见

每多一个智能体，就多一份协调、观测与成本——拆分要有明确收益

03 常用架构模式特点对比

模式	核心思想	优势	挑战	典型场景
分层	按职责纵向分层，单向依赖	结构清晰、技术成熟	性能损耗、整体部署	企业级业务系统
事件驱动	构件通过事件异步通信	高可扩展、松耦合	调试难、最终一致	订单履约、消息流水线
微内核	最小核心 + 插件扩展	核心稳定、易扩展	插件治理复杂	产品化应用、IDE
微服务	按业务拆分自治服务	独立部署演化、易伸缩	分布式复杂性	大型互联网平台
虚拟化/云	应用与资源解耦、弹性伸缩	按需伸缩、成本可控	云依赖、冷启动	流量波动大的系统
管道-过滤器	数据流经多级过滤器单向加工	易组合复用、可并行高吞吐	不适合交互、传输有开销	编译器、ETL、流处理
智能体（新）	大模型驱动、工具调用、自主协作	能力涌现、自然语言交互	不确定性、成本、可信	智能助手、自动化流程

实践中常组合使用：如"分层 + 微服务 + 云"是当前企业应用的主流组合；智能体架构则常作为微服务体系中的一种新型服务形态存在

03 架构反模式：这些"坑"比选错模式更常见

大泥球

没有清晰边界，任何模块都能访问任何模块，改一处崩三处

破法：先画依赖图，找出环，按业务能力切第一刀

分布式单体

拆成了服务，却仍共享数据库、同步串行调用、必须一起发布

破法：要么退回单体，要么真正做数据自治与契约解耦

金色锤子

团队只会一种技术，什么问题都用它砸：什么都上微服务/什么都上大模型

破法：先明确质量目标，再比较候选方案，允许"不引入新技术"

过度设计

为"可能的未来"预先抽象：三层缓存、五种消息队列、万能插件框架

破法：按当前与可预见的需求设计，保留扩展点而非实现扩展

烟囱系统

每个业务竖一条独立系统，数据重复、口径不一，集成靠人导表

破法：建立统一数据模型与集成层，按领域规划而不是按部门规划

无主架构

没人对整体结构负责，谁需要谁加，架构在"自然演化"中腐化

破法：明确架构负责人，建立评审与决策记录机制

反模式的共同根源：结构跟着"当下最急的事"长，而不是跟着"业务边界与质量目标"长

04

架构设计过程

确定架构原型 → 智能体时代的设计要点

通用架构原型

按应用类型选择

LLM 网关与记忆管理

编排框架选型

04 架构设计过程：属性驱动设计（ADD）五步

1. **确认架构攸关需求**
   找出真正影响结构的质量属性场景与约束
2. **选择模式与战术**
   为每个场景挑选候选方案与实现手段
3. **划分模块与接口**
   确定构件边界、职责与对外契约
4. **分配职责与视图**
   把职责落到构件与节点，产出视图
5. **评估与迭代**
   用场景走查，识别风险并修正

为什么不直接"照着最佳实践画"

最佳实践回答的是"别人怎么做"，ADD 回答的是"在你的约束下该怎么做"。没有场景与优先级，模式选择就只是模仿。

迭代而非瀑布

每轮只处理少量高优先级场景，产出可验证的片段架构；随着认知加深回填细节，而不是一次画出全部。

作业 4 的架构设计部分，就按这五步组织你的设计说明

04 第一步：把"要好用"变成可检验的场景

质量属性场景的六要素

刺激源	刺激	制品	环境	响应	度量
谁发起	发生了什么	作用于什么	什么条件下	系统怎么做	做到什么程度

示例：大促期间（环境），用户（刺激源）集中下单（刺激），订单服务（制品）应在 200ms 内返回结果，且错误率低于 0.1%（响应与度量）

从需求里"挖"出 ASR

- ? 这句话背后要求的是性能、可用还是可修改？
- ? 有没有可量化的指标？没有就追问到有
- ? 与哪些约束冲突？冲突处就是权衡点
- ? 把所有 ASR 列出来，按业务价值排序

三类约束同样要收集

技术约束
必须用集团统一的中台、必须兼容旧数据库

组织约束
团队只有 6 人、运维不承接自建机房

法规约束
数据不得出境、日志留存 6 个月

04 第二步：先选战术，再选模式

战术是"解决某个质量属性的具体手段"，模式是"把战术组织起来的结构"。战术决定下限，模式决定协作方式。

质量属性	常见战术	常用模式
性能	资源池化、并发、缓存、异步、读写分离、批处理	分层 + 云弹性 + 缓存层
可用性	冗余、心跳、故障转移、熔断、限流、降级	微服务 + 多可用区部署
可修改性	关注点分离、接口抽象、依赖倒置、插件化	分层 / 微内核 / 微服务
安全性	认证、授权、加密、审计、输入校验、最小权限	网关 + 纵深防御
可测试性	契约测试、依赖注入、可观测埋点	分层 + 服务虚拟化
成本	按需伸缩、缓存命中、模型路由、批量化	云原生 + LLM 网关

常见误区：先定了"我们要上微服务"，再去找理由——正确顺序是先明确场景，再看哪种结构最能满足它

04 第三、四步：划分模块、定义接口、落到视图

划分模块

- 以业务能力/限界上下文为主轴
- 以质量属性场景校正边界（是否需要独立伸缩/独立部署）
- 明确每个模块的职责与"不做什么"
- 检查依赖方向，消除环

产出：模块清单 + 依赖图 + 职责说明

定义接口

- 接口是契约：输入、输出、错误、幂等性、版本
- 同步用 REST/gRPC，异步用事件 Schema
- 接口设计面向"使用者的场景"，而非内部实现
- 明确 SLA：超时、重试、限流配额

产出：接口清单 + 关键接口定义 + 事件目录

分配职责与画视图

- 职责矩阵：每个场景由哪些模块协作完成
- 逻辑视图：构件 + 关系 + 关键接口
- 运行视图：进程/线程/消息与并发点
- 部署视图：节点、副本、网络与容灾

产出：至少逻辑视图 + 部署视图 + 一个场景走查

判断划分是否合理：能否只改一个模块就完成一个典型需求变更?

04 第五步：评估与迭代——架构是"走"出来的

轻量评估四件事

场景走查
拿 3~5 个关键场景，在白板上"走"一遍，看是否真的能实现

找风险点
哪些场景可能不满足？依赖哪些尚未验证的技术？

找权衡点
哪个决策同时影响多个质量属性？代价是什么？

原型验证
对高风险点做一次技术验证（性能压测、模型效果评测）

评估之后的动作

- **接受**
  场景满足，写入决策记录，进入详细设计
- **增强**
  补充战术或调整模式，重新走查
- **降级目标**
  与业务方重新协商质量目标（要有依据）
- **记录遗留风险**
  明确风险、监控方式与触发预案

架构评审不是"批斗会"：目标是尽早暴露风险，而不是证明谁对

迭代节奏建议：每个迭代留出"架构回顾"时间，把新出现的场景补进评估清单

04 通用架构原型：从单机到云原生与智能体

1	**单机应用**	界面、业务与数据同在一台设备	记事本、单机游戏
2	**客户 / 服务器（C/S）**	客户端负责界面，数据库服务器负责数据	传统桌面管理系统
3	**三层结构（C/S/S）**	客户端 + 应用服务器 + 数据库，逻辑上移	经典企业应用
4	**分布式多层 + 缓存**	网关 + 服务集群 + 缓存 + 数据库	大型互联网平台
5	**云原生**	容器编排 + 微服务 + 弹性 + 可观测	华为云上的现代应用
6	**智能体应用原型（新）**	对话入口 + LLM 网关 + 编排 + 工具 / RAG + 记忆	智能助手、AI 编码助手

演化主线：节点越来越多、耦合越来越松、弹性越来越强——智能体原型不是替代，而是叠加在云原生之上的新型骨架

04 按应用类型确定架构原型

Web 应用（前后端分离）

SPA / SSR 混合渲染；关注首屏性能、SEO、状态管理

移动端应用

端上轻逻辑 + 云端 API；关注弱网、离线与多端一致性

桌面与跨端应用

Electron / 鸿蒙等；本地能力、自动更新、多端协同

面向服务的应用

API 优先、以服务对外提供能力；关注契约、治理与版本

数据与 AI 应用

批流一体数据管道 + 模型服务；关注吞吐、特征与算力调度

智能体应用（新）

以对话为入口、大模型为引擎；关注上下文、工具、记忆与可控性

04 六类应用原型：不同形态，不同骨架

应用类型	前端形态	后端形态	架构要点
Web 应用	SPA / SSR 混合（React、Vue、Next）	BFF + 微服务 + CDN	首屏性能、渲染方式取舍、状态管理、前后端契约
移动端应用	原生 / 跨平台（Flutter、ArkUI）	移动网关 + BFF + 微服务	弱网与离线、包体与启动速度、端侧缓存、灰度发布
桌面与跨端	Electron / WPF / 鸿蒙应用	本地服务 + 远程 API	自动更新、本地与服务端数据同步、系统能力调用
面向服务	多端复用同一服务能力	API 网关 + 服务注册与编排	服务契约治理、版本兼容、SLA 与限流
数据与 AI	可视化大屏 / 分析工作台	批流一体管道 + 模型服务（Serving）	吞吐与时效取舍、特征管理、算力与成本调度
智能体应用	对话界面 / 内嵌 AI 入口	LLM 网关 + 编排 + 工具 / RAG + 记忆库	上下文与记忆管理、工具治理、可信与成本控制

原型不是"简化版架构"，而是"经过验证的起点"：先套用原型，再按本项目的质量场景做增删；同一产品常是多种原型的组合

04 智能体时代的设计要点（一）：网关、上下文与记忆

LLM 网关与模型路由

- 统一接入点：认证、限流、计费、日志集中管理
- 模型路由：简单任务用小模型，复杂任务用大模型，控制成本
- 多模型容灾：主备模型切换，避免绑定单一供应商
- 响应缓存与批处理：相同输入直接命中缓存

上下文与记忆管理

- 上下文窗口有限：会话历史需要压缩、摘要、淘汰
- 分层记忆：短期（会话内）· 长期（向量库持久化）
- 检索式记忆：按语义召回相关历史与知识，注入上下文
- 状态外置：会话状态放外部存储，服务实例可随时替换

为什么是新的架构关注点

- 传统架构管"数据结构"，智能体架构还要管"提示词与上下文"
- 上下文即运行时资产：丢失、污染都会直接导致输出劣化
- 多轮对话的"有状态性"打破了无状态服务的默认假设
- 提示词需要版本管理，与代码同等的工程待遇

04 智能体时代的设计要点（二）：编排与选型

编排框架选型

轻量自研	直接调模型 API + 自写循环，透明可控
编排框架	LangGraph 等图式编排，支持分支、并行、回溯
多智能体框架	AutoGen、CrewAI 等，角色化协作
低代码平台	可视化拖拽编排，业务人员可参与

选型决策要点

- 复杂度匹配：能不用框架就不用——框架会屏蔽细节、增加调试成本
- 可观测性优先：每一步的输入输出可追踪、可回放
- 工具生态：是否支持 MCP 等标准协议，决定扩展成本
- 锁定风险：框架抽象层是否可替换，避免深度绑定

> 与传统选型一致的道理：框架解决 80% 通用问题，剩下的 20% 个性化部分决定成败

部署策略（一）：集中式部署

集中式部署



- 除后台数据库外，所有功能与层次部署在一个节点
- 简单、成本低，应用间不需要通信
- 单点失效风险：节点故障即服务中断
- 适合内部系统、演示系统、用户量小的场景

分布式部署要额外考虑什么？

通信	协议选型、序列化、超时与重试
事务	跨节点数据一致性（最终一致 / 补偿）
安全	节点间认证、传输加密
可用	健康检查、故障转移、容量规划

05 部署策略（二）：分布式多层部署

经典多层部署模式



- 客户-服务器结构：两层，简单直接
- 多层结构：按功能或逻辑分层部署在不同节点
- 层与层之间是网络调用——部署图决定性能与可靠性基线

智能体应用的部署形态（新）

- 智能体服务以微服务形态部署在云上，与业务服务同构治理
- 模型网关独立部署：统一管控 Token 成本与限流
- 向量数据库 / 知识库作为新型基础设施单独部署
- 异步长任务（长循环推理）需要队列与超时治理
- 本课程实践：应用与模型服务均部署在华为云

05 云上部署：多可用区、弹性与灰度

用户入口　CDN（静态资源就近分发）→ WAF / DDoS 防护 → 全局负载均衡（按地域/健康状态分流）

可用区 A	可用区 B	可用区 C
应用集群（容器）	应用集群（容器）	应用集群（容器）
服务实例 ×N	服务实例 ×N	服务实例 ×N
缓存主节点	缓存从节点	缓存从节点
数据库主库	数据库只读副本	数据库只读副本
区内负载均衡	区内负载均衡	区内负载均衡
日志与监控 Agent	日志与监控 Agent	日志与监控 Agent

弹性
按 CPU / QPS / 队列长度自动扩缩容，设最小与最大副本数

灰度与回滚
按流量比例或用户标签放量，指标异常自动回滚

容灾
跨区副本 + 定期演练；明确 RTO / RPO 指标

数据合规
数据驻留地域、加密与密钥管理、审计日志留存

云上部署的核心不是"上云"，而是把"失败是常态"作为设计前提

05 部署决策：五个问题定方案

1 **规模与增长预期?**	小规模稳定业务：集中式即可；快速增长：预留水平扩展能力	决定是否分布式
2 **可用性要求几级?**	允许小时级停机：单区；要求 99.99%：多可用区 + 自动故障转移	决定副本与容灾
3 **数据能否出境 / 出区?**	受监管数据必须本地驻留，跨区只同步必要摘要	决定部署地域
4 **团队运维能力如何?**	无专职运维：优先托管服务 / Serverless，少自建中间件	决定自建还是托管
5 **成本上限与计费模式?**	稳定负载用包年包月，波峰负载用按量 + 弹性	决定资源采购

部署方案要写进架构文档：节点清单、副本数、伸缩策略、备份与恢复演练计划，缺一不可

根据架构与部署策略选择技术路线

应用框架	Spring Boot / Spring Cloud、.NET、Django 等
前端框架	Vue、React、小程序等
数据库	关系型、文档型、缓存、向量数据库
中间件	消息队列、注册中心、网关、任务调度
模型服务	大模型 API / 私有化部署、模型网关

选型原则

- 匹配架构约束，而不是追逐技术热点
- 团队熟悉度优先：用得熟比用得新更重要
- 生态与社区活跃度决定长期维护成本
- 警惕锁定：优先选可替换的标准化组件
- 成本显性化：许可证、云资源、Token 都是真金白银
- AI 辅助选型：可让 AI 对比方案，但决策依据要自己验证

06 实例：某系统逻辑设计与技术选型

高层逻辑设计：数据采集分析系统

数据采集 **数据存储** **数据同步** **数据分析**

▼ 服务编排与治理层 ▼

API 网关 · 消息队列 · 调度中心

▼ 基础设施层 ▼

容器集群 · 监控告警 · 日志追踪

逻辑设计先于技术选型：先定"有哪些块、如何协作"，再定"用什么实现"

对应技术选型（示例）

数据采集	消息队列 Kafka / 设备接入
数据存储	关系库 + 时序库 + 对象存储
数据同步	分布式任务调度
数据分析	批处理引擎 + AI 模型服务
部署运行	华为云容器集群 + 监控体系

06 智能体技术选型：单智能体与多智能体的技术栈

选型顺序 先定形态（单 / 多）→ 再选编排框架 → 定模型与推理方式 → 定工具协议 → 定记忆与检索 → 最后补可观测与治理

单智能体：一条清晰的栈

应用框架	LangChain / LlamaIndex、Spring AI、Semantic Kernel、Dify / Coze
编排内核	LangGraph（图/状态机）、OpenAI Agents SDK、Claude Agent SDK
模型	GPT / Claude / Gemini / Qwen / DeepSeek / GLM；私有化用 vLLM、SGLang
工具协议	MCP、 Function Calling、OpenAPI 自动注册
记忆检索	pgvector / Milvus / Qdrant / ES、Mem0、Redis
可观测	Langfuse、LangSmith、OpenTelemetry GenAI

起步建议：LangGraph 或 Agents SDK + MCP + pgvector，一周内可跑通闭环；低代码场景用 Dify / Coze

多智能体：多了四类组件

编排框架	LangGraph 多节点、AutoGen、CrewAI、MetaGPT、AgentScope
协作协议	A2A（智能体间）、MCP（智能体→工具）、消息队列 Kafka / NATS
共享状态	Redis / PostgreSQL / 对象存储；任务板与中间产物
隔离执行	E2B、Firecracker、K8s Job；每角色独立沙箱与配额
长流程引擎	Temporal、Camunda：可靠重试、补偿、人工审批
治理观测	统一网关、OPA 策略、跨智能体 Trace（Langfuse）

提醒：多智能体的复杂度主要来自"协调"，不是"智能"；先解决状态与超时，再谈角色设计

团队熟悉度 Java 团队优先 Spring AI；Python 团队可用 LangGraph

可控 vs 快 要可控选开源 + 私有化；要快选托管平台与现成 Agent SDK

成本上限 按 token 单价 × 预估调用量倒推模型档位与缓存策略

06 智能体系统案例与选型策略

系统	形态	关键架构选择	可借鉴点
GitHub Copilot / Cursor	单智能体 + 检索	代码库索引与语义检索、编辑器内联工具、diff 确认后落盘	检索质量比模型大小更决定体验；写操作必须人工确认
Claude Code	单智能体 + 工具集	文件/终端/搜索工具 + 计划模式 + 分级权限；项目级上下文文件	工具描述与权限分级是核心设计；长任务要能中断与续跑
Manus	多智能体 + 沙箱	规划者 + 执行者分工，任务在虚拟机沙箱中执行，状态持久化	隔离执行与状态持久化，让长任务可恢复、可审计
MetaGPT	多角色流水线	产品/架构/开发/测试角色，按 SOP 传递结构化产物	角色 = 职责 + 输入输出契约，而不是"给个名字"
企业客服智能体	RAG + 工具 + 转人工	知识检索、订单/工单工具、置信度阈值触发人工接管	置信度与升级策略，比追求全自动更现实

按确定性选 流程稳定→固定工作流；步骤开放→ReAct；子任务可并行→多智能体

按风险选 高风险动作（资金、外发、删除）加审批与沙箱；低风险可全自动

按规模选 先单体上线收集 badcase，出现上下文瓶颈或角色冲突再拆分

反模式：一上来就搭"全自动多智能体团队"——角色没有明确产出物、没有终止条件、没有评测集，最后只剩一堆无法复现的对话记录

005

通用质量属性设计

性能 · 可靠性 · 安全 · 智能体时代的质量新命题

定义与分类

性能与负载均衡

可靠性与安全

智能体质量属性

005 定义质量属性

运行时可见

用户可感知的运行特征

- 性能：响应时间、吞吐量
- 可用性：无故障运行时间
- 安全性：访问控制与防护
- 易用性：交互效率与容错

演化时可见

面向开发与维护的特征

- 可修改性：变更成本
- 可移植性：跨环境迁移
- 可测试性：缺陷发现能力
- 可复用性：构件复用度

业务侧

面向商业与运营的特征

- 时间到市场：上市速度
- 成本：开发与运行成本
- 可伸缩：随业务成长
- 合规：法规与标准符合性

质量属性相互制约：架构设计的本质是在冲突的质量属性间做权衡

05 质量属性的分类与度量：先分清，才能设计

运行时质量（系统跑起来才看得见）

性能	响应时间、吞吐量、资源占用
可用性	正常运行时间占比、故障恢复时间
可靠性	无故障运行时长、错误率
安全性	抗攻击能力、权限边界、审计完整性
易用性	任务完成率、学习成本

设计时质量（改代码时才感觉得到）

可修改性	改一个需求要动几个模块
可测试性	能否隔离验证、用例覆盖成本
可复用性	构件能否在别的系统直接使用
可演化性	引入新技术是否需要重写
可部署性	发布是否可灰度、可回滚

度量的三条要求

① 可量化：给出数字与单位，而不是"要快"

② 可测量：明确在什么负载、什么数据规模下测

③ 可归责：指标对应到具体构件与负责人

不能用数字表达的期望，往往会在验收时变成扯皮

005 通用质量属性清单

性能	响应时间、吞吐量、并发用户数	策略：缓存、异步、纵向/横向扩展
可用性	系统无故障运行的时间比例（如 99.99%）	策略：冗余部署、故障转移、降级
安全性	认证、授权、数据保密与完整性	策略：身份认证、加密、最小权限
可修改性	变更一个功能所需的成本与风险	策略：分层解耦、依赖倒置
可伸缩性	负载增长时平滑扩容的能力	策略：无状态设计、弹性伸缩
可测试性	发现缺陷、验证功能的难易度	策略：自动化测试、可注入依赖
可观测性	运行状态可度量、问题可定位	策略：日志、指标、链路追踪
可信性（新）	AI 输出正确、可溯源、可纠错	策略：幻觉控制、引用溯源、人工审核

05 设计时质量：为"以后要改"而设计

可修改性

一个需求变更要动几处？

- 按业务能力划分模块
- 接口隔离变化点
- 配置化替代硬编码
- 依赖倒置

可测试性

能否不启动全系统就验证？

- 依赖注入与外部适配层
- 契约测试与测试替身
- 关键路径可观测埋点
- 无状态化便于重放

可复用性

这块能力别处能直接用吗？

- 能力与场景解耦
- 稳定的公共契约
- 可独立发布的构件包
- 避免业务专属逻辑下沉

可演化性

换技术栈要重写多少？

- 防腐层包裹外部依赖
- 模型与存储解耦
- 版本化接口
- 灰度并行运行

设计时质量不会在演示中体现，但会在"第 3 年的维护成本"里体现——这是架构师最该坚持的部分

性能问题：两类扩展与负载均衡

两类解决方案

纵向扩展（Scale Up）

提升单机性能：更强 CPU、更大内存；有物理上限，成本非线性增长

横向扩展（Scale Out）

增加服务器数量，分流请求；需要架构支持无状态与水平复制

其他手段：缓存热点数据、读写分离、异步削峰

负载均衡策略



结合健康检查：自动摘除故障节点

05 性能实战：双十一与亿级网关的高并发设计

场景数字（公开数据）

· 双十一订单创建峰值：约 58 万笔 / 秒

· 核心 API 网关：日均调用千亿级、峰值 QPS 亿级

· 热点商品：瞬时读 QPS 数百万

通用设计思想

· 漏斗式削减：每层只放行下一层能承受的流量

· 尽量靠前拦截：能在 CDN 挡住就不进网关，能缓存就不查库

· 同步转异步：排队、削峰，用"最终一致"换"不崩"

五层手段（自上而下）

接入层	CDN + 页面静态化；限流（令牌桶 / 漏桶）；WAF 防护；无效流量前置拦截
网关层	网关集群 + 动静分离；鉴权与路由前置；按租户 / 接口分级限流熔断
服务层	无状态水平扩展；消息队列异步削峰；降级与熔断保核心链路
数据层	多级缓存 + 热点探测；读写分离；分库分表；单元化部署就近容灾
演练层	全链路压测定容；预案演练；限流阈值按大促节奏动态调整

对应战术：资源管理（并发处理、缓存）· 调度与仲裁（负载均衡、排队）· 数据冗余（读写分离、分片）

05 缓存设计：位置、模式与三个坑

缓存放哪儿

浏览器 / 客户端	静态资源、列表页快照；成本最低
CDN	图片、视频、前端包；就近分发
API 网关	热点接口结果；统一失效
应用进程内	字典、配置；注意多实例不一致
分布式缓存	会话、热点数据；Redis 等
数据库	查询缓存、物化视图

三种读写模式

旁路缓存

读：先缓存后库并回填；写：更新库再删缓存（最常用）

写穿透

写时同时更新缓存与库，读永不落空，写代价高

写回

先写缓存异步落库，性能最好但有丢数据风险

三个必须防的坑

穿透（查不存在的数据）→ 空值缓存 / 布隆过滤器

击穿（热点 key 过期）→ 互斥重建 / 逻辑过期

雪崩（大量 key 同时失效）→ 过期时间加随机抖动

缓存是"用一致性换性能"的交易：先想清楚能容忍多旧的数据，再决定缓存多久

05 分布式缓存与智能体记忆（Agent Memory）

分布式缓存：今天的主流形态

Redis Cluster	16384 槽分片 + 主从复制 + 自动故障转移，水平扩展读写
多级缓存	本地缓存（Caffeine）+ 分布式缓存，热点 key 本地兜底
热点探测	实时统计热 key，推送到本地缓存或独立热点集群
一致性	延迟双删、订阅 binlog（Canal）失效缓存，容忍短暂不一致
高可用	哨兵 / 集群模式、跨机房复制、缓存预热与降级预案

Agent Memory：智能体的"缓存" 新技术

工作记忆	上下文窗口内的对话与中间结果；容量有限，需要裁剪与压缩
长期记忆	向量库 / RAG：把历史对话、文档、经验 Embedding后检索召回
分层结构	情景记忆（发生过什么）、语义记忆（知道什么）、程序性记忆（会怎么做）
写入策略	会话摘要压缩、重要度打分、遗忘与合并（如 Mem0、Letta 类方案）
新问题	记忆污染与越权召回：检索质量决定对错，需要溯源、隔离与审计

从"缓存数据"到"缓存经验"：记忆是智能体的缓存——但检索错一条记忆，比缓存脏读更危险

可靠性问题：冗余策略

核心思路：冗余

- 可靠性 = 确保系统持续提供可靠服务、避免失效
- 单点失效是可靠性最大的敌人——消除一切单点
- 冗余的代价：资源成本翻倍、数据一致性复杂化
- 配套机制：健康检查、故障自动转移、备份恢复演练

典型冗余部署

应用层 节点1	缓存层 节点1	数据库 节点1
应用层 节点2	缓存层 节点2	数据库 节点2
多实例 + 负载均衡	主从 / 集群	主备 / 双活 + 异地备份

失效切换由监控与调度自动完成：感知故障 → 摘除节点 → 流量转移到健康节点

05 容错模式：让故障"局部化"

熔断器的三个状态

关闭 正常放行 ⇄ **打开** 直接拒绝 ⇄ **半开** 少量试探

关闭 → 错误率超阈值 → 打开 → 冷却后转半开 → 试探成功回关闭，失败再打开

熔断保护的是调用方：宁可快速失败，也不要被下游拖死

另外五种常用手段

限流	令牌桶 / 漏桶 / 滑动窗口；保护自身不被压垮
降级	关闭非核心功能，保住主流程（如推荐位为空）
隔离	线程池 / 连接池分组（舱壁），避免一个依赖耗尽全部资源
超时与重试	分级超时预算；重试必须配合退避与上限
幂等	重试与重复投递的前提；用唯一键或状态机保证

容错不是"永不出错"，而是"出错时影响可控、可恢复、可解释"

005 安全问题：Web 架构中的安全防线

安全问题贯穿多个层面：认证（Authentication）· 授权（Authorization）· 保密性 · 完整性 · 不可否认性

用户 / 终端	网络传输	Web 服务器	应用服务	数据库
客户端安全	传输加密（HTTPS）	WAF / 边界防护	认证与授权	数据加密与审计

- 每一层都要设置防线：单点防护（如只靠防火墙）在纵深攻击面前不堪一击
- 典型风险：注入攻击、越权访问、敏感数据泄露、拒绝服务攻击
- 安全是"非功能需求"中最不能妥协的一项——出事的代价远超防护成本

005 通用安全模式：各类角色访问模式

基于角色 / 声明的访问控制

普通用户 ▶ 只读自己的数据

业务操作员 ▶ 读写授权业务数据

系统管理员 ▶ 管理用户与配置

- 认证解决"你是谁"，授权解决"你能做什么"
- 最小权限原则：只授予完成任务所必需的权限
- 集中认证（单点登录）+ 细粒度授权是主流实践

智能体时代的安全新课题

- 提示注入：用户或外部内容诱导智能体执行恶意指令
- 工具滥用：智能体获得的工具权限必须受限并审计
- 数据外泄：上下文中携带的敏感信息流向外部模型的风险
- 对策：输出过滤、权限沙箱、敏感信息脱敏、操作审计

05 通信、消息与状态管理：服务之间怎么说话

通信：同步还是异步

同步调用
REST / gRPC；
实时、简单，但强依赖、易级联失败

异步消息
MQ / 事件；
解耦、削峰，但最终一致、调试难

流式输出
SSE / WebSocket；
大模型逐字返回、长连接

决策依据：能否容忍等待？失败后能否重试？

消息设计四要素

幂等
消费端按消息 ID 去重，重投不产生副作用

顺序
同一业务键路由到同一分区，保证局部有序

可靠
发送确认 + 持久化 + 死信队列 + 退避重试

可追踪
消息头带 traceId，贯穿生产与消费

状态往哪里放

无状态优先
实例不保存会话，任意实例可处理任意请求

状态外置
会话与进度存 Redis 或数据库，实例可替换

断点续传
长任务把执行状态落库，失败从检查点恢复

会话粘滞
仅在无法外置时使用，且要有失效预案

架构落地质量的分水岭，往往不在"分几层"，而在这些通信与状态的细节约定

005 智能体时代的质量属性（一）：可信性

为什么"可信"成为一等质量属性

- 传统软件：输出由代码逻辑决定，对错可静态验证
- 智能体软件：输出由模型生成，存在幻觉与不确定性
- 同一输入可能得到不同输出——正确性从"属性"变成"概率"
- 架构必须为"可验证、可纠错、可追责"而设计

架构层面的对策

- RAG 溯源：回答必须引用知识来源，无据不答
- 双重校验：模型输出经过规则校验或第二模型复核
- 人工审核点：高风险操作（如扣款、发通知）必须人确认
- 降级策略：模型不可信时退回确定性业务逻辑
- 输出契约：结构化输出 + 校验器，不合格即重试

对照传统：性能靠缓存、可靠靠冗余、可信靠"冗余验证"——思路一脉相承

005 智能体时代的质量属性（二）：成本、时延与可观测

成本（Token 成本）

- 按 Token 计费：成本随用户量线性增长
- 长上下文、多轮循环 = 成本放大器
- 对策：模型分级路由、结果缓存、上下文压缩、批处理折扣

时延

- 大模型推理秒级起步，远慢于传统毫秒级调用
- 智能体多步循环使时延成倍叠加
- 对策：流式输出改善体感、并行工具调用、小模型前置分类

可观测性

- 记录每一次调用的提示词、上下文、输出与成本
- 链路追踪覆盖"模型调用-工具调用-人工环节"全流程
- 回放能力：出问题时能重现当时上下文
- 这是智能体运维（AIOps）的基础设施

005 智能体时代的质量属性（三）：安全与可评测

安全

- 提示注入与越狱：输入即"代码"，需输入过滤与意图识别
- 工具权限沙箱：智能体的每个工具调用最小授权、全程审计
- 数据边界：敏感数据脱敏后才可进入模型上下文
- 合规：生成内容标识、训练数据版权、个人信息保护

可评测性（Evals）

- 传统回归测试基于确定断言，模型输出"软性"对错需要新方法
- 建立评测集：典型问题 + 参考答案 + 评分标准（规则或模型评分）
- 提示词、模型、RAG 参数的每次调整都要跑评测集——像代码一样做回归
- 质量门禁：评测不达标不上线，与单元测试同等的工程地位

一句话：把"对模型质量的度量"当作架构的一部分来设计

05 质量属性权衡：没有"全都好"的架构

敏感点
影响某一个质量属性的决策
例：加缓存 → 性能↑

权衡点
同时影响多个属性的决策
例：加缓存 → 一致性↓

风险点
可能不满足场景的决策
例：单库写入成瓶颈

非风险点
已确认满足、可放心的决策
例：静态资源走 CDN

三组经典权衡，面试与答辩都绕不开

一致性 vs 可用性	分布式写入	强一致需跨节点协调，故障时不可用；最终一致可用性高但有延迟窗口	按业务定：资金强一致，点赞可最终一致
性能 vs 可维护性	缓存与冗余数据	缓存提升读性能，却引入失效与一致性维护成本	先量化收益，再决定缓存层级与粒度
安全 vs 体验	多因子认证与频繁校验	校验越多越安全，但步骤增加、转化率下降	按风险分级：敏感操作强校验，浏览弱校验
成本 vs 冗余度	多副本与多活	副本越多可用性越高，资源与运维成本同步上升	按 RTO/RPO 定副本数，不做"过度多活"

架构评审的价值：把权衡点显式写出来，让业务方参与决策，而不是架构师独自背锅

05 质量属性落地：从场景到可验证的技术方案

1. 质量属性场景 › 2. 选战术 › 3. 定模式 › 4. 落到产品与技术 › 5. 定度量与验证

质量属性	典型战术	可落地的技术与产品	度量指标
性能	并发、缓存、异步、资源调度	Nginx / OpenResty、Sentinel、Redis Cluster、Kafka、ShardingSphere、K8s HPA	P95/P99 时延、QPS、缓存命中率
可用性	冗余、心跳、熔断、降级、多活	K8s 多副本、Resilience4j、Keepalived、同城双活、Chaos Mesh 演练	可用性 99.99%、RTO/RPO、MTTR
安全	认证、授权、加密、审计、隔离	OAuth2 / OIDC、Keycloak、Vault、WAF、OPA、mTLS	漏洞数与修复时长、拦截率、越权事件
可修改性	模块化、契约、特性开关	DDD 限界上下文、API 网关、Nacos / Apollo 配置、契约测试	变更前置时间、回归通过率
成本	弹性伸缩、配额、分级服务	HPA / VPA、Serverless、Spot 实例、LLM 网关模型分级	单请求成本、资源利用率

落地三问 这个战术由哪个组件承担？故障时怎么降级？用什么指标证明它生效？

05 实例：电商大促与智能体系统的落地清单

场景 双十一 0 点：峰值 58 万笔/秒订单、P99 时延 < 200ms、可用性 99.99%、故障 30 分钟内恢复

层次	关键组件与技术	承担的质量属性
接入层	CDN + WAF + Nginx/OpenResty 限流；DNS 智能调度；静态资源预热	性能、安全
应用层	K8s 多副本 + HPA 弹性；Sentinel 熔断降级；本地缓存 + Redis Cluster 多级缓存；热点探测与隔离	性能、可用性
数据层	分库分表（ShardingSphere）；读写分离；Kafka 削峰填谷；最终一致 + 对账补偿	性能、一致性
运维层	全链路压测与容量评估；Chaos Mesh 故障演练；SkyWalking/Prometheus 可观测；预案与一键降级	可用性、可运营

同一套方法 换成智能体系统：质量目标变成"首 token 时延、单任务成本、工具成功率、人工介入率"，战术与技术同样要逐项落到组件

质量目标	技术手段与产品	度量指标
响应快	LLM 网关流式输出 + 语义缓存 + 小模型前置分类；就近部署推理节点	首 token P95 < 1.5s
成本可控	模型分级路由（小模型优先）+ 上下文压缩 + 结果缓存 + 预算熔断	单任务成本 < 0.3 元
结果可信	RAG 引用溯源 + 结构化输出校验 + 关键动作人工确认 + 评测集回归	工具成功率 > 99%、人工介入率 < 10%

小结 本讲知识地图：一条从需求到结构的路

①	**软件设计**	把需求翻译成可实现的方案；四项内容：架构、构件、接口、数据
②	**架构设计**	构件 + 关系 + 原则；架构是一组难改的决策，用 4+1 视图描述
③	**架构模式**	分层 / 事件驱动 / 管道-过滤器 / 微内核 / 微服务 / 云 / 智能体：每种都有适用条件与代价
④	**设计过程**	确认 ASR → 选战术与模式 → 划模块定接口 → 落视图 → 评估迭代
⑤	**质量属性**	运行时与设计时质量；性能、可靠性、安全、可维护与智能体特有属性

贯穿全讲的一句话

架构的本质是"在约束下做取舍"：模式是工具，质量属性是目标，决策记录是证据。

08 作业4：架构设计（15分）

任务1（5分）：架构调研分析

- 选择一个真实系统（或课程案例系统）调研其架构
- 分析其采用的架构模式、部署策略与技术选型
- 给出架构图与关键质量属性的分析

任务2（10分）：系统架构设计

- 基于前期需求模型，完成系统的架构设计
- 选择并论证架构模式、部署策略与技术选型
- 绘制架构视图

AI 使用要求

- 允许使用 AI 辅助资料调研与方案对比
- 须声明使用环节与方式，并列出关键提示词
- 架构决策的论证与权衡必须体现小组自己的思考
- 答辩重点考察设计决策的理由——AI 生成 ≠ 理解